

Group Project Phase 3: UI Integration

Objective: Finalize the implementation of the multi-tenant travel agency platform.

A baseline UI providing basic search features is provided. Your team's responsibility is to extend the provided codebase to fully incorporate and integrate the backend APIs your team developed in Phase 2.

Part 1: Environment Setup & UI Integration

1. Initialize the Repository

- a. Create and navigate to your project directory:

```
mkdir your-project  
cd your-project
```

- b. Clone the provided baseline repository:

```
git clone https://github.com/chungsiksong/travel-agency-platform.git
```

2. Start the Backend Server

Ensure your team's backend API server is fully running before attempting to launch the UI.

Two APIs have been added

- a. Retrieve saved bookings for a specific agent and user combination.
GET /api/v1/bookings/by-agent-user
- b. Authenticate a user by email and password. Returns the User_ID on success.
GET /api/v1/users/login

3. Configure and Start the UI

- a. Navigate to the root folder of the travel-agency-app.
- b. To support local multi-tenant routing, append the necessary host entries to your system. Run the following command:

For Mac/Linux:

```
sudo sh -c 'printf "\n127.0.0.1 agenta.local\n127.0.0.1 agentb.local\n" >> /etc/hosts'
```

For Windows:

1. Open Notepad as an Administrator (Search for Notepad, right-click -> "Run as administrator").
2. Open the file located at: C:\Windows\System32\drivers\etc\hosts
3. Add the same lines to the bottom: 127.0.0.1 agenta.local 127.0.0.1 agentb.local
4. Save the file.

- c. Install dependencies and start the application:

```
npm run dev
```

- d. Open your browser to test the tenant-specific views:

- Agency A: <http://agenta.local:5173>
- Agency B: <http://agentb.local:5173>

Part 2: Authentication & Login

4. Access the UI and select either Travel Agency A or B.
5. To test the login functionality, seed your local database with a test user:

```
INSERT INTO Users (First_Name, Last_Name, Email, Phone_Number)
VALUES ('John', 'Doe', 'john.doe@example.com', '555-123-4567');
```

6. Authenticate using the following credentials:

Account Name: john.doe@example.com

Password: CMPE-131@2026

Part 3: Core Implementation Requirements

3.1 Make a Travel Plan

- The UI must allow users to search for travel options based on your team's specific scenarios.
- The application must consume the APIs developed during Phase 2.

- Validation: The search results displayed in the UI must be strictly consistent with the JSON payloads returned directly from your backend APIs.

3.2 Save Booking

- The application must support saving reservations for both Airline tickets and Hotels (minimum requirement).
- Extend the saved data to include any additional information dictated by your team's custom scenarios.

3.3 View and Edit Bookings

- Users must be able to retrieve and review their saved bookings.
- The UI must provide functionality to edit an existing booking. Upon submission, the UI must refresh to display the correctly updated results.

Deliverables & Grading Rubric

Your final submission for Phase 3 must include the following artifacts. Please note the grading multiplier: the Code Score acts as a binary multiplier. If the submitted code does not compile or run, the Code Score is 0, resulting in a total phase grade of 0. If it runs successfully, the Code Score is 1.

Deliverable	Format	Maximum Points
Source Code (Your team's version of travel-agency-api and travel-agency-app)	Code Repository	Multiplier (0 or 1)
Flight Search Results (Meeting criteria based on your team's scenarios.)	UI Screenshot	30 points
Hotel Search Results (Meeting criteria based on your team's scenarios.)	UI Screenshot	30 points
Saved Booking Details (Displaying a successfully saved reservation.)	UI Screenshot	40 points

Total Possible Points: 100